

KATHMANDU UNIVERSITY

SCHOOL OF ENGINEERING

Department of Electrical & Electronics Engineering

PROJECT REPORT



DESIGN AND IMPLEMENTATION OF AUTONOMOUS MAZE SOLVING ROBOT

*A Third year final project report submitted in partial fulfillment
of the requirements for the degree of
Bachelor of Engineering*

by:

Sakriya Dahal (32028)

Digant Gurmachhan (32063)

Bardan Khadka (32033)

Ajar Shrestha (020026)

February 4, 2025

CERTIFICATION

THIRD YEAR PROJECT REPORT ON DESIGN AND IMPLEMENTATION OF AUTONOMOUS MAZE SOLVING ROBOT

by:

Sakriya Dahal (32028)

Digant Gurmachhan (32063)

Bardan Khadka (32033)

Ajar Shrestha (020026)

Approved by:

1. Project Supervisor

(Signature)

(Name)

(Date)

2. Head of the Department

(Signature)

(Name)

(Date)

ABSTRACT

In robotics, autonomous movement is an important feature that enables the robot to move independently from one point to another. Autonomous movement within an unknown area requires the robot to carry out investigations. The concept of path finding has an important place in the field of robotics and is based on one of the most important areas of robotics, the Path Planning Algorithm. Implementing the algorithm, a maze solving robot that uses a flood fill algorithm was designed, through real time investigation using sensors to find the path through a $n \times m$ maze. The efficiency of Flood Fill algorithm combined with the accuracy provided by associated hardware enabled through microcontrollers (ESP32 and arduino) resulted in a functioning maze solving robot capable of operating autonomously.

Keywords: *Maze solving robot, path planning algorithm*

ACKNOWLEDGMENT

We extend our sincere appreciation to the Department of Electrical and Electronics Engineering at Kathmandu University for their generous support and provision of essential resources. Our gratitude is also extended to our project supervisor, Mr. Aayush Bista for his expert guidance, continuous encouragement, and valuable insights that significantly influenced the project's direction. His and the departments' commitment and collaboration have been instrumental in achieving the milestones until now as outlined in this report. We look forward to continued success as we work towards our project goals.

LIST OF FIGURES

Figure 3.1: ESP32	6
Figure 3.2: L298N Motor Driver.....	7
Figure 3.3: N20 motor with encoder.....	7
Figure 3.5: Maze solving robot prototype 1.....	9
Figure 3.6: Maze solving robot prototype 2.....	9
Figure 4.1: Simulation of Flood Fill Algorithm.....	11
Figure 4.2: Experimental setup of motor control with ESP32 and L298N.....	12
Figure 4.3: 3D printing motor holders.....	13
Figure 4.4: Maze M1 for testing.....	14

LIST OF TABLES

Table 1: Gantt Chart.....	8
---------------------------	---

TABLE OF CONTENTS

ABSTRACT.....	3
ACKNOWLEDGMENT.....	1
LIST OF FIGURES.....	2
LIST OF TABLES.....	3
TABLE OF CONTENTS.....	5
CHAPTER I: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Problem Definition.....	2
1.3 Objectives.....	2
1.4 Significance of study.....	2
CHAPTER II: TECHNOLOGY AND LITERATURE SURVEY.....	4
CHAPTER III: PROJECT METHODOLOGY.....	5
3.1 System Overview.....	5
3.1.1 Component Selection and Integration.....	5
3.1.2 Components Used.....	6
i. ESP32	6
ii. Motor Driver L298N.....	6
iii. N20 Motor with Encoder.....	7
iv. Ultrasonic Sensor.....	8
3.1.3 Hardware Implementation.....	8
3.1.4 Algorithm Implementation.....	10
3.1.5 Integration and Testing.....	10
CHAPTER IV: RESULT AND ANALYSIS.....	11
4.1 Simulation of Flood Fill algorithm.....	11
4.2 Experimental Setup.....	11
4.2 Work Accomplishments.....	12
4.4 Next Steps.....	14
CHAPTER V: CONCLUSION.....	15
Gantt Chart.....	16
REFERENCES.....	17
APPENDIX.....	18

CHAPTER I: INTRODUCTION

1.1 Background

Robotics has emerged as a crucial field intersecting various disciplines, ranging from engineering to artificial intelligence. With the rise of automation and the increasing demand for smart systems, the study of robotics has become more important. The project at hand focuses on developing a self-navigating robot for solving mazes. This project not only serves as a practical application of robotics principles but also provides valuable insights into autonomous systems.

At its core, this project aims to deepen understanding of how robots operate and move within complex environments. By simulating real-world challenges through maze-solving tasks, students gain hands-on experience in designing, building, and programming autonomous systems. This practical learning approach bridges the gap between theoretical concepts and practical implementation, equipping students with essential skills for future work in robotics.

The significance of this project extends beyond academic exploration. As robotics becomes increasingly prevalent in various industries, the ability to develop autonomous systems capable of efficient navigation is highly valuable. From warehouse logistics to search and rescue operations, autonomous robots offer great opportunities to improve processes and enhance productivity.

Furthermore, the project aligns with the broader goals of electronics engineering education. By involving students in projects that integrate hardware design, sensor integration, and algorithm development, educators promote critical thinking and problem-solving skills crucial for success in the rapidly changing technological landscape. Through collaborative efforts and innovative solutions, students are empowered to address real-world challenges and advance the field of robotics.

The idea of maze solving micromouse was introduced by IEEE Spectrum Magazine in 1977 and the first contest was held in 1979. The first non-graphical wall following algorithm using micromouse was developed in 1999. In 2008, Chang Yuen Chung, a student of Universiti Teknologi Malaysia (UTM) designed a micromouse using Flood Fill Algorithm. It is one of the graph theory maze solving algorithms.

Over time, the Micromouse competition has progressed, pushing the limits of self-navigation and inspiring many generations of robotics enthusiasts. Nowadays, Micromouse robots use advanced sensors, powerful microcontrollers, and sophisticated algorithms to navigate intricate mazes with remarkable accuracy and efficiency.

The maze-solving robot project takes inspiration from this rich history of Micromouse competitions, giving students an opportunity to contribute to the tradition of self-navigation. By designing and constructing their maze-solving robot, students will not only gain practical experience in robotics but also become part of a respected legacy of innovation and exploration.

1.2 Problem Definition

The primary problem identified for this project is developing a robot that can autonomously navigate through a maze while finding the optimized and efficient path to reach the end goal. This involves integrating various components, such as sensors, motors, and microcontroller, and implementing algorithms for maze exploration and path planning.

Furthermore, the project aims to optimize the robot's movement while minimizing costs and resource requirements. Balancing functionality with budget constraints poses an additional challenge, requiring careful selection of components and efficient utilization of resources.

1.3 Objectives

The objective of the maze-solving robot project is:

- To develop an autonomous maze-solving robot capable of navigating through complex maze environments.
- To implement efficient pathfinding algorithms that enable the robot to identify the optimal routes to the goal.
- To gain a comprehensive understanding of robotics principles and mechanisms through hands-on experimentation and practical application.

1.4 Significance of study

The project holds significant importance for robotics and engineering education. With this project, students are able to bring together their understanding of electronics and programming and merge them together in order to design and implement an autonomous

maze-solving robot. Through the design and construction of an autonomous robot capable of efficiently navigating mazes, students gain practical experience in implementing complex systems, bridging the gap between theoretical concepts and real-world applications. Moreover, the project serves as a platform for skill development, fostering critical thinking, problem-solving, and teamwork skills essential for success in the engineering field. Ultimately, the significance of this study lies in its dual impact: advancing robotics research and preparing students for future challenges in engineering and technology.

CHAPTER II: TECHNOLOGY AND LITERATURE SURVEY

The field of robotics has witnessed remarkable advancements in the development of autonomous maze-solving robots. Various researchers and engineers have contributed to this area, employing different algorithms and techniques to enhance the capabilities of these robots.

Semuil Tjiharjadi, from the Computer Engineering Department at Maranatha Christian University in Bandung, Indonesia, introduced a maze-solving robot capable of autonomously navigating through unknown environments. Equipped with ultrasonic range finders, the robot utilized the flood-fill algorithm and the pledge algorithm to explore and map the maze. Results from experiments demonstrated an 80% success rate in finding the shortest path to the destination point. [1]

Musfiqur Rahman, from the University of Liberal Arts Bangladesh, developed an ESP32-based autonomous maze solver robot. By employing simple algorithms such as the wall-following algorithm and flood-fill algorithm, this robot was designed to efficiently navigate through mazes. The project involved hardware and software development, with testing conducted on a 4x4 maze to verify the robot's capability to find the shortest path. [2]

Akib Islam, Farogh Ahmad, and P. Sathya, from the School of Electronics Engineering at VIT University in Tamilnadu, India, presented a maze-solving robot capable of automatic navigation in unknown areas. Utilizing wall-following techniques based on LSRB and RSLB algorithms, the robot integrated inputs from ultrasonic sensors, infrared sensors, and wheel rotation encoders to determine the shortest path through the maze. The robot demonstrated the ability to learn arbitrary mazes and find optimal routes for solving them, making it suitable for navigational purposes[3].

Researchers worldwide have contributed to this area, employing various algorithms and techniques to enhance the capabilities of these robots. These advancements include the utilization of ultrasonic range finders and algorithms for navigation, as well as the integration of sensors for automatic navigation in unknown areas. Such progress highlights the potential for practical applications in diverse domains.

CHAPTER III: PROJECT METHODOLOGY

3.1 System Overview

Our system integrates basic hardware components such as motors, drivers, wheels, and sensors along with a microcontroller put together onto a chassis and the microcontroller is programmed with a suitable path finding algorithm, motor movement and sensory input. The microcontroller (ESP32) shows real-time decision-making and precise navigation allowed by two N20 geared motors with rubber wheels driven by a L298N motor driver and a free caster wheel at the front. Equipped with Infrared sensor modules (LM-393) and motor encoders(magnetic), the robot perceives its environment, avoids obstacles, and optimizes pathfinding. Other components of the system are the backbone of the robot in order to operate it as a standalone system during the solving process.

3.1.1 Component Selection and Integration

The maze-solving robot project began by carefully selecting and integrating its components. DC motors were chosen for their ease of use and mounted onto a simple chassis, providing the robot with locomotion. A wooden chassis was employed to securely house all components, offering options for customization if needed. The ESP32 microcontroller served as the brain of the robot, controlling its actions based on sensor input. Infrared sensors will be utilized to detect walls and obstacles, while encoders will measure the robot's position and distance traveled through motors' rotation, essential for precise navigation and mapping.

3.1.2 Components Used

i. ESP32

ESP32 is a series of low-cost, low-power system-on-chip microcontrollers with integrated Wi-Fi and dual-mode Bluetooth. The ESP32 series employs either a Tensilica Xtensa LX6 microprocessor in both dual-core and single-core variations, an Xtensa LX7 dual-core, or a single-core RISC-V microprocessor and includes built-in antenna switches, RF balun, power amplifier, low-noise receive amplifier, filters, and power-management modules.

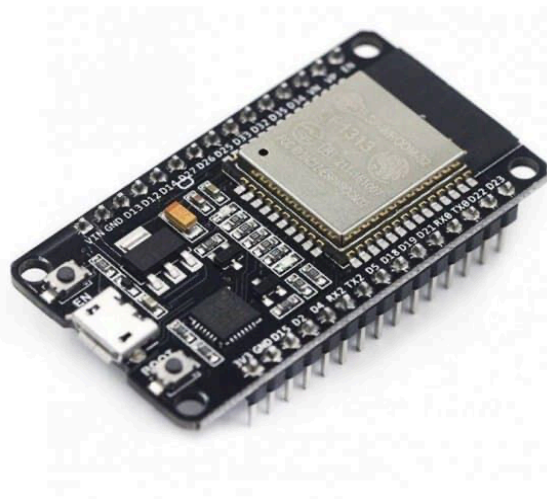


Figure 3.1: ESP32

ii. Motor Driver L298N

The L298N module is a dual H-bridge high-power motor driver module. It can control up to 4 DC motors, or 2 DC motors with directional and speed control at the same time. The power supply that the motor driver needs to operate properly is defined by this voltage range. It ought to be in line with the voltage that powers the DC motor. The size and power of the DC motor that the driver can control depend on the maximum current it can manage. By controlling current flow, the H-bridge arrangement enables the driver to adjust the speed and direction (forward or reverse) of the DC motor.

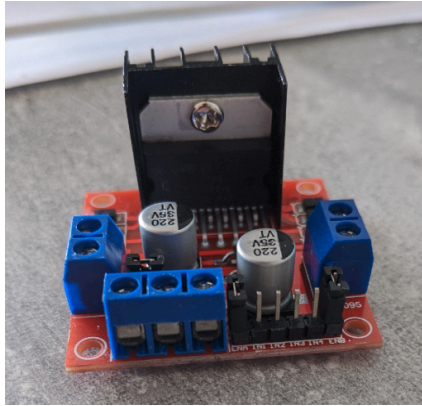


Figure 3.2: L298N Motor Driver

iii. N20 Motor with Encoder

N20 motor is a compact, lightweight and powerful geared DC motor that provides high torque output and low rpm performance. It is very reliable and efficient. It is useful for driving wheels that carry high loads. A motor encoder is a device that measures the position and speed of an electric motor's shaft and converts that information into electrical signals.

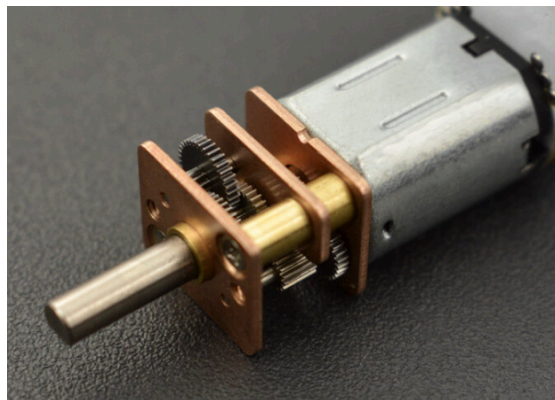


Figure 3.3: N20 motor with encoder

iv. Ultrasonic Sensor

Ultrasonic sensors are devices that generate or sense ultrasound energy. They can be divided into three broad categories: transmitters, receivers and transceivers. Transmitters convert electrical signals into ultrasound, receivers convert ultrasound into electrical signals, and transceivers can both transmit and receive ultrasound.



Figure 3.4: Ultrasonic sensor module [5]

3.1.3 Hardware Implementation

Firstly, the brain of the system, RISC-V included in the ESP32 development board allows us to control all the other hardware implemented in this project. The is run as a standalone system by providing power through the 5V constant port of the L298N motor driver module which is further powered by the two 18650 batteries in series connection. The I/O ports are connected to the motor driver in order to send motor control inputs to control the motors simultaneously to control the movement of the robot. With the help of the three Infrared sensors, the system is able to identify surrounding walls and move around each cell of the maze in a controlled manner. The magnetic encoder provides the microcontroller with real-time electrical signals of the physical rotation of the wheels which allows calculation of movement of mouse between cells.

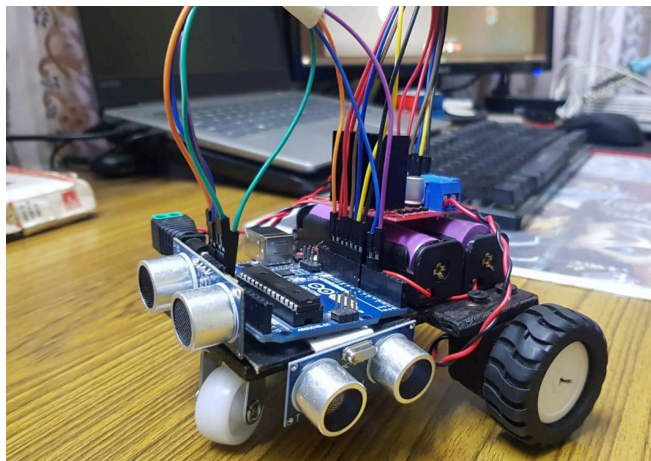


Figure 3.5: Maze solving robot prototype1

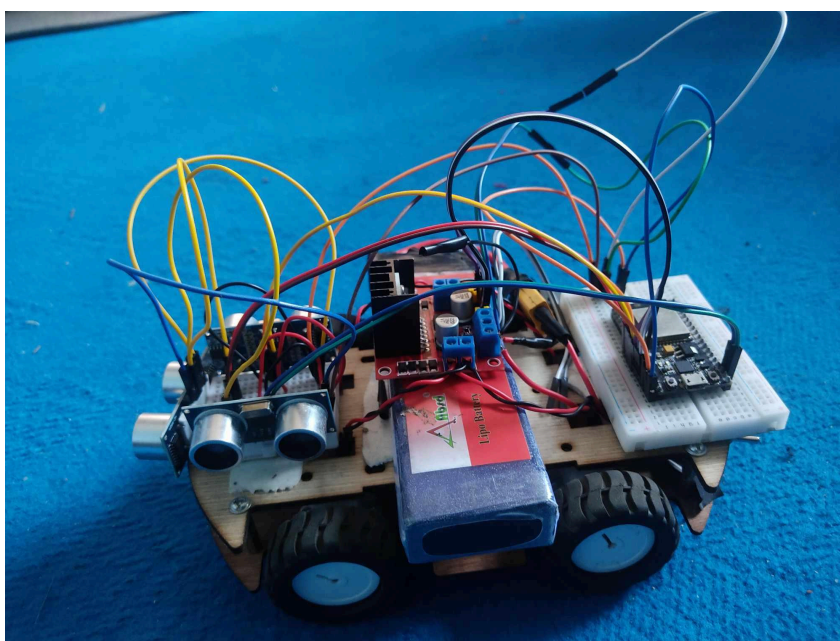


Figure 3.6: Maze solving robot prototype2

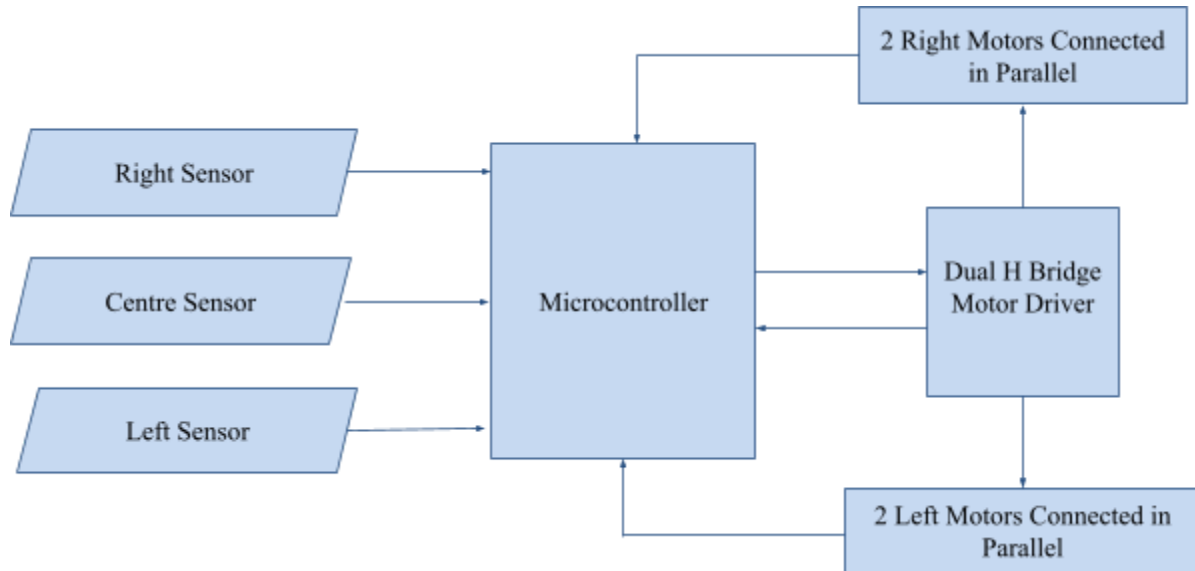


Figure 3.7: Block Diagram of Robot Architecture

3.1.4 Algorithm Implementation

Programming was undertaken on a development board to execute navigation algorithms, interpret sensor data, and control motor movement. Control algorithms were employed for precise motor speed regulation and obstacle avoidance, ensuring smooth navigation through the maze. Mapping algorithm was implemented to guide the robot towards the maze's center, utilizing encoder data to track its position and progress. To solve the maze we used the flood fill algorithm.

The flood fill algorithm for maze-solving robots systematically explores the maze by starting from the initial position and traveling through accessible cells. It initializes a grid to represent the maze and a queue to store cells for exploration. Beginning from the starting position, the algorithm marks each visited cell as explored and checks adjacent cells for accessibility. If an adjacent cell is accessible and unexplored, it marks it as explored and adds it to the queue for further exploration. This process continues until all accessible cells are explored. Movement occurs step by step based on the path obtained from backtracking, if needed. This process is repeated until it reaches the destination.

3.1.5 Integration and Testing

Once hardware and software components were assembled, they were integrated to create a cohesive system, ensuring proper communication and functionality. The maze-solving robot

will undergo rigorous testing to validate its performance, including maze navigation, obstacle avoidance, and mapping accuracy where the robot receives input data from sensors, these obtained data are analyzed in a flood fill algorithm which provides appropriate directions for actuators. This process is repeated cell by cell until the target cell is reached. Simulations using tools will be conducted to validate and optimize the robot's behavior before real-world testing.

CHAPTER IV: RESULT AND ANALYSIS

4.1 Simulation of Flood Fill algorithm

A simple simulation of the Flood fill algorithm solving an arbitrary maze in the form of a 5×5 matrix was run. Each term of the matrix is an ordered pair that consists of an 8-bit number representing the wall states of the cell and a digit representing the cell's distance from the target cell at the center ie. (wall states, distance).

These data would change overtime as the simulated micromouse traversed through the arbitrary maze until the target cell is mapped.

<pre> front:0 ----left:0 ----right:0 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8(X) 4,4 North ----- left front:0 ----left:0 ----right:0 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7(X) 8 4,3 West ----- forward front:0 ----left:0 ----right:0 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6(X) 7 8 4,2 West ----- forward </pre>	<pre> front:0 ----left:0 ----right:0 -- 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5(X) 6 7 8 4,1 West ----- forward front:0 ----left:0 ----right:0 -- 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4(X) 5 6 7 8 4,0 West ----- right front:0 ----left:0 ----right:0 -- 0 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3(X) 4 5 6 7 4 5 6 7 8 3,0 North ----- forward </pre>	<pre> front:0 ----left:0 ----right:0 ---- 0 1 2 3 4 1 2 3 4 5 2(X) 3 4 5 6 3 4 5 6 7 4 5 6 7 8 2,0 North ----- forward front:0 ----left:0 ----right:0 ---- 0 1 2 3 4 1(X) 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 1,0 North ----- forward front:0 ----left:0 ----right:0 ---- 0(X) 1 2 3 4 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 0,0 North ----- end </pre>
Figure 4.1: Simulation of Flood Fill Algorithm		

4.2 Experimental Setup

All the testing and experimentation was formerly done in a breadboard with the components connected with jumper wires. The components were connected using datasheets from reliable sources and the testing of hardware and software programs were conducted before implementing them onto the base of the robot.

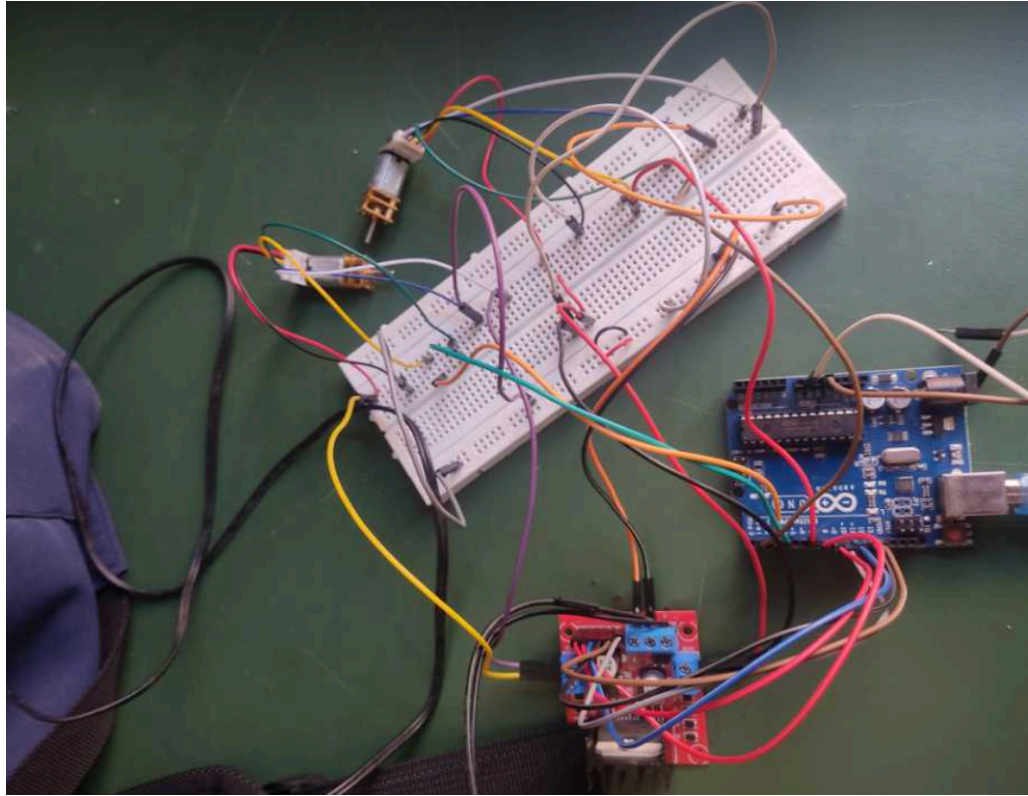


Figure 4.2: Experimental setup of motor control with Arduino and L298N

4.2 Work Accomplishments

- As planned, the project was undertaken with the two halves, hardware and software, being worked on parallelly to ensure efficiency of the work.
- On the hardware side, the various equipment were tested and configured as per the need of the project. This included the ESP32, motor driver, motors and the ultrasonic and infrared sensors on board.
- The ultrasonic sensors used initially were replaced by infrared sensors. This decision was made in light of the fact that IR sensors have a narrower line of sight and are easier to implement for that reason. However due to some issue with obtaining accurate signal from IR sensors, the focus shifted back to ultrasonic sensors.
- Simultaneously, the ESP32 program for basic movements of the maze solving robot was written using the Arduino IDE, alongside the Flood fill algorithm's implementation in the form of a simple grid simulation in the IDE terminal.

- The robot was assembled on a wooden base for integrating the hardware and software halves, and to test the working of the robot. Plywood was chosen for the lightweight nature of the wood veneer which would cause minimal additional load on the motors.
- Initially, the motor holders were carved of wood as well. This, along with uneven weight distribution, caused the robot to be slightly unstable. The robot would bump into walls. Thus it was opted to 3D print the motor holders instead.



Figure 4.3: 3D printing motor holders

- After making sure the robot's locomotory functions are operating, a test maze was designed to check whether the Flood fill algorithm was working in conjunction with the ultrasonic sensors.

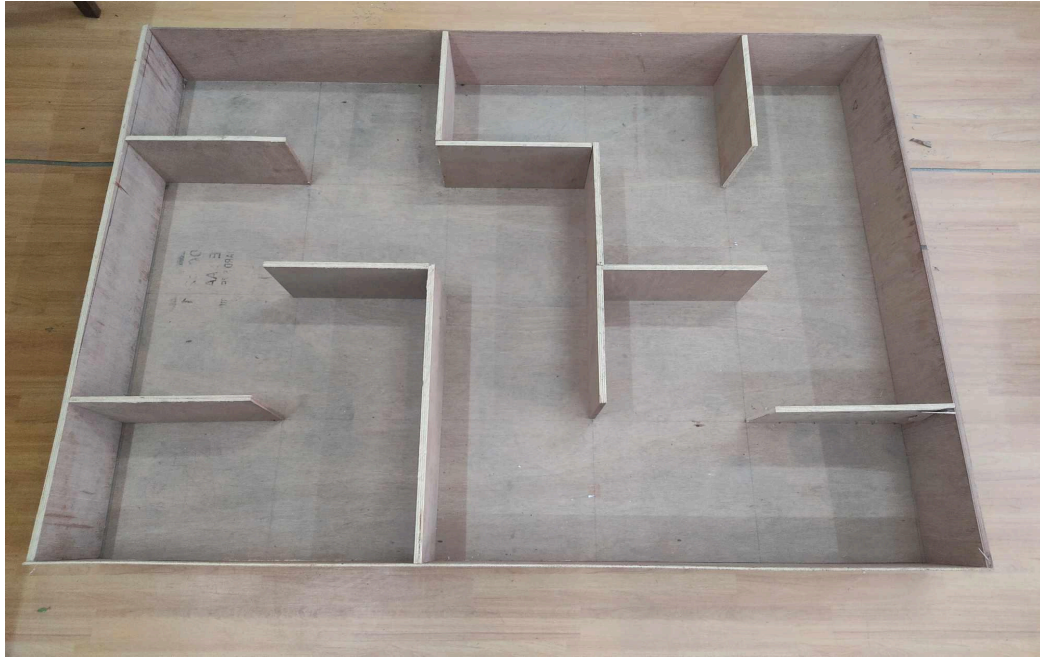


Figure 4.4: Maze M1 for testing

- It was made sure that the robot is taking data from the three ultrasonic sensors and updating the matrix representation of the maze by using the flood fill algorithm on set intervals as it traverses through the maze. It is necessary that the data is updated every time the position and state/orientation of the robot changes in the maze.
- Optimized the robot and tried to minimize errors and inconsistency.

4.4 Next Steps

- Using better PID with encoders to ensure more stable and accurate operation of the micromouse.
- Using TOF sensors instead of ultrasonic sensors to map the maze.
- Using better motor and wheels along with custom PCB to increase robusticity and compactness.

CHAPTER V: CONCLUSION

This project successfully developed an autonomous maze-solving robot using the Flood Fill algorithm, which demonstrated the potential of combining algorithmic pathfinding with precise hardware integration. The robot efficiently navigated a structured maze environment using ultrasonic sensors, successfully reaching its goal within an optimized timeframe. The transition from simulation to real-world implementation confirmed the systems' capability in autonomous navigation, showcasing the importance of fine-tuning both hardware and software components to ensure reliable performance.

Looking ahead, a significant next step would involve compressing and minimizing the existing project to create a more compact, high-performance robot suitable for competitive environments, such as the Micromouse competition. This could be achieved by designing a custom PCB, utilizing smaller components, and optimizing the layout for both space and power efficiency. By chipping away at the bulk and reducing size, the robot would not only become more agile but also potentially gain a competitive edge in speed and efficiency. This approach would allow the project to evolve into a more sophisticated and capable system, capable of competing in robotic competitions that demand precision, agility, and adaptability.

Gantt Chart

Year	2024										2025	
Month	Mar	Apr	May	Jun	July	Aug	Sep	Oct	Nov	Dec	Jan	Feb
Literature review												
Proposal												
Component Analysis												
Simulation												
Hardware Implementation												
Integration and Testing												
Project Demonstration												

Table 1: Gantt Chart

REFERENCES

- [1] S. Tjiharjadi, "Design and Implementation of Flood Fill and Pledge Algorithm for Maze Robot", International Journal of Mechanical Engineering and Robotics Research, 2019 [Online]. Available:
<https://api.semanticscholar.org/CorpusID:220363004>
- [2] M. Rahman, "AUTONOMOUS MAZE SOLVING ROBOT," 2017. doi: 10.13140/RG.2.2.34525.82403.
- [3] A. Islam, F. Ahmad, and S. Pichandi, "SHORTEST DISTANCE MAZE SOLVING ROBOT," International Journal of Research in Engineering and Technology, vol. eISSN pISSN, pp. 253–259, Jul. 2016.
- [4] "Curio Res." Accessed: Nov. 13, 2024. [Online]. Available:
<https://curiores.com/positioncontrol>
- [5] L. Levkoff and P. Dokter, "SparkFun Electronics ProtoSnap and repurposed electronics," in *Proceedings of the Sixth International Conference on Tangible, Embedded and Embodied Interaction*, Kingston Ontario Canada: ACM, Feb. 2012, pp. 381–382. doi: 10.1145/2148131.2148223.

Appendix:

```
#include<ArduinoQueue.h>

#include "flood_fill.hpp"
//east north west south convention
//check for the array

//visted array : at each iteration to know if the cell is already
visited
//visited array must be cleared at each iteration
bool visted[5][5]; // this visited thingy should be made 0 in the start
of every iteration !!!not done !! do in .ino file

byte maze_arr[5][5] = {0};
byte wall_arr[5][5] = {0};
bool running = true;

//actual flood fill array for the mouse in alpha version
void set_flood_fill() {

    memset(visted, 0 , 5 * 5 * sizeof(visted[0][0]));
    memset(maze_arr, 0 , 5 * 5 * sizeof(maze_arr[0][0]));

    // wall_arr[4][0] = 0b1101;
    // wall_arr[3][0] = 0b0101;
    // wall_arr[2][0] = 0b0110;
    // wall_arr[3][1] = 0b0101;
    // wall_arr[2][1] = 0b0001;
```

```

        ArduinoQueue<m_point> q(20);
        m_point point(DEST_X, DEST_Y);
        q.enqueue(point);
        while ( !q.isEmpty()){
            m_point temp = q.dequeue();

            int a = temp.x , b= temp.y;

            if ( !visted[a][b] && in_boundary(temp) ){// for the cell
that is being processed
                visted[a][b] = 1; // if cell is processed it is visitedd
                m_point point_arr[4] ={m_point(a, b-1, West) ,
m_point(a , b+1 ,East), m_point(a-1, b, North), m_point(a+1, b, South)};
//cells in 4 direction of the cell begin processed
                for ( auto elem : point_arr){ //possible optimization ?
dont know
                    if ( in_boundary(elem) && !visted[elem.x][elem.y]){
// are the cells around the cell begin processed in boundary and not
already processed
                        if ( !((wall_arr[a][b] & elem.dir) ||
(wall_arr[elem.x][elem.y] & inverse_dir(elem.dir))) ) { // & and && are
different things , to check if the point has walls or not to insert
into the queue
                            maze_arr[elem.x][elem.y] = (maze_arr[a][b] +
1); //increment the value
                            q.enqueue(elem); // push into the queue for
processing
                        }
                    }
                }
            }
        }
    }
}

```

```

//finding the direction to which the mouse will move
m_point closest_neighbour_dir(mouse_state m1){
    int a = m1.x , b = m1.y ;

    m_point point_arr[4] ={m_point(a, b-1, West) , m_point(a , b+1
,East), m_point(a-1, b,North), m_point(a+1, b, South)};
    m_point point_arr1[4];
    int c = 0 ; // c is the new count for the array
    //disqualification loop
    for(int i = 0 ; i< 4 ; i++){

        if( (in_boundary(point_arr[i])  && !((point_arr[i].dir &
wall_arr[a][b]) || (inverse_dir(point_arr[i].dir) &
wall_arr[point_arr[i].x][point_arr[i].y] ))){//to only select cubes
which are inside boundary and has no wall
            //smallest = point_arr[i]
            //the whole thing can be done in this rather than in two
loops but not checked
                // if ( maze_arr[smallest.x][smallest.y] >
maze_arr[point_arr1[i].x][point_arr1[i].y] ){
                    // smallest = point_arr1[i];

                // }

            point_arr1[c] = point_arr[i] ;
            c++;
        }

    }

    // no solution condition
    if (c == 0 || (maze_arr[m1.x][m1.y] == 0 && m1.x != 2 && m1.y!=2))
{ // no direction to move to from above loop and the value given by the
flood fill is 0 in value other than (2,2)

    running = false;
    Serial.print("no solution end");
    Serial.println();
    return m_point(m1.x , m1.y , m1.dir);
}

```

```

    // for finding smallest
    m_point smallest = point_arr1[0];
    for( int i = 1 ; i < c ; i ++){
        if ( maze_arr[smallest.x][smallest.y] >
maze_arr[point_arr1[i].x][point_arr1[i].y] ){
            smallest = point_arr1[i];
        }
    }

    return smallest;
}

//utility functions
// for checking if the point is inside the boundary or not
inline bool in_boundary(m_point point ) {
    return point.x >= 0 && point.x <= 4 && point.y >= 0 && point.y <=
4 ;
}

//rotate left algorithm
direction rotate_dir(direction dir1 , int rotate_by){
    return (direction)((dir1 << rotate_by) | ( dir1 >> (4 - rotate_by)) )
& 0b00001111;
}

// to invert the direction of mouse
direction inverse_dir(direction dir){
    return rotate_dir(dir , 2 );
}

//helper function to print direction enum
void printDir(direction dir)
{
    switch (dir) {
        case East:
            Serial.print("East");
            break;
        case West:

```

```

        Serial.print("West");
        break;
    case North:
        Serial.print("North");
        break;
    case South:
        Serial.print("South");
        break;
    default:
        Serial.print("error");
    }
}

//helper function for the print function
void printBinary(byte inByte)
{
    for (int b = 7; b >= 0; b--)
    {
        Serial.print(bitRead(inByte, b));
    }
}

// function to debug and print the array
void print_array(const mouse_state &m1 ){
    for(int i = 0 ; i <5 ; i++){
        for ( int j = 0 ; j < 5; j ++ ){
            printBinary(wall_arr[i][j]);
            Serial.print('\t');
            Serial.print( maze_arr[i][j]);
            Serial.print('\t');

        }
        Serial.print( "\n");
    }
    Serial.print(m1.x);
    Serial.print(",");
    Serial.print(m1.y);

```

```
Serial.println();  
printDir(m1.dir);  
Serial.println();  
  
Serial.print("-----  
-----");  
Serial.print( "\n");  
  
}
```